

Lecture 4: Process scheduling

Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF
GOTHENBURG

What to read (main textbook)

- Chapter 2.4, 8.1.1, 8.1.2, 8.1.4, 10.3.4, 11.4.1

(extra facultative reading: 5.1-5.7, 1.10 from Silberschatz
Operating System Concepts)

Objectives

- Get deeper into processes, threads and their scheduling / execution
- Discuss different types of systems (batch/interactive/real-time)
- Discuss challenges of multi-processor/multi-core architectures

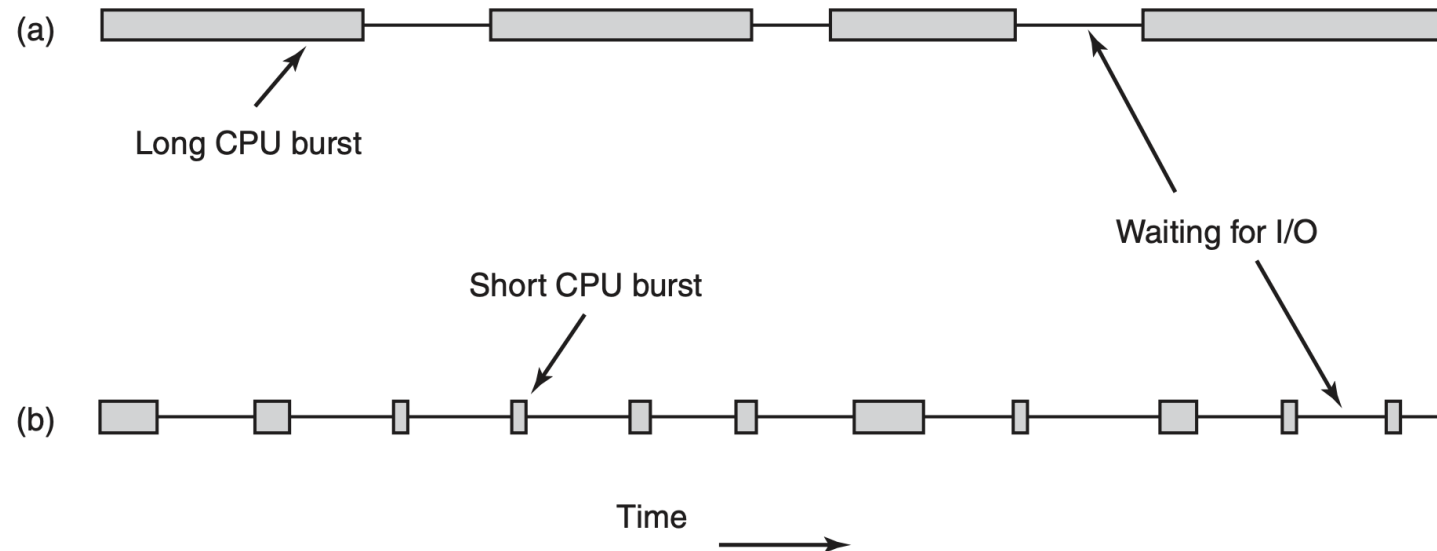
Agenda

- Introduction
- Batch / Interactive / Real-time systems scheduling
- Processes vs. Threads scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Agenda

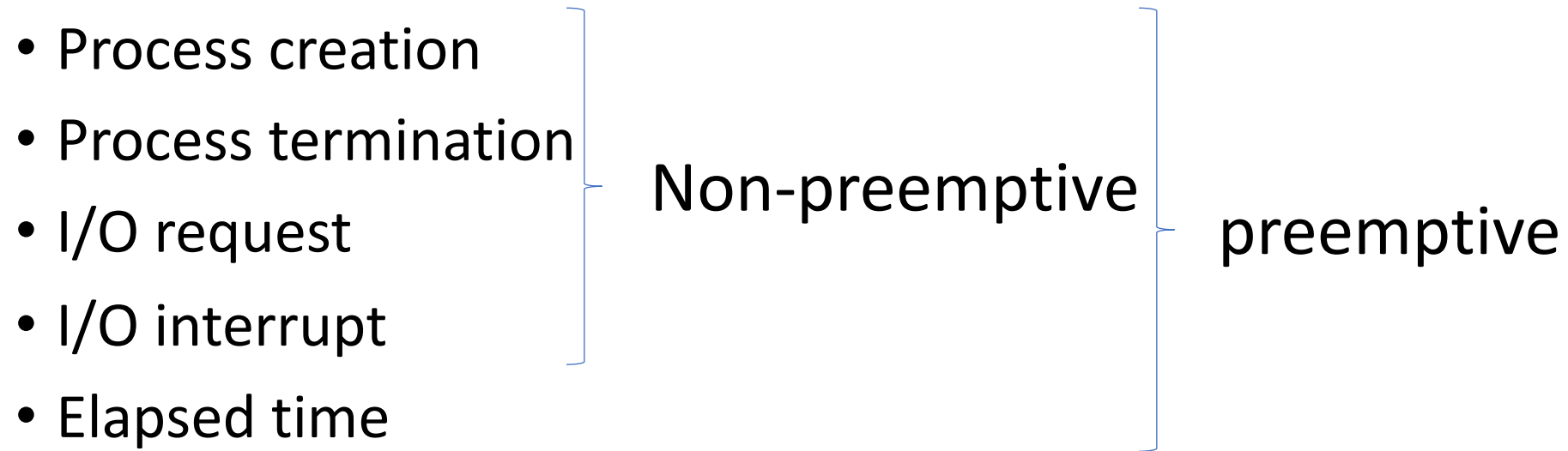
- Introduction
- Batch / Interactive / Real-time systems scheduling
- Processes vs. Threads scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Introduction



- 2 types of processes:
 - a) CPU-bound (or compute-bound)
 - b) I/O bound
- Notice: I/O does not mean I/O takes a lot, it means few CPU cycles in-between I/O calls

When can the OS take scheduling decisions?

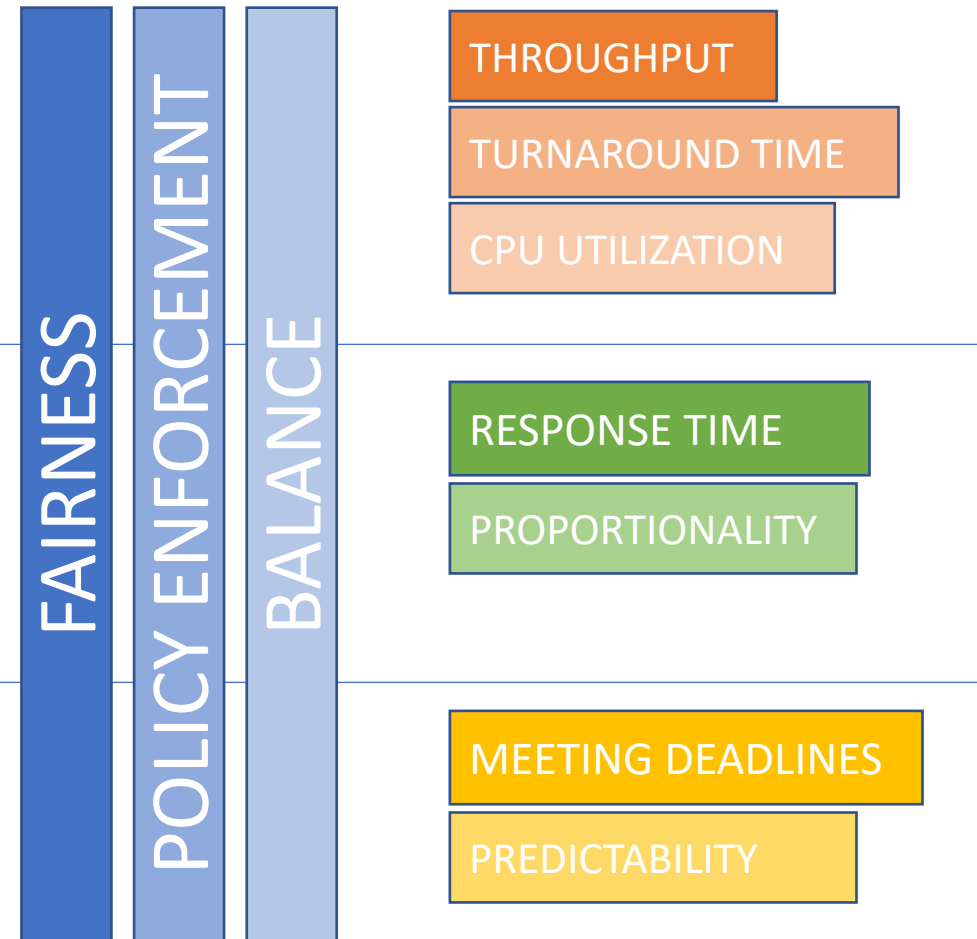


Agenda

- Introduction
- Batch / Interactive / Real-time systems scheduling
- Processes vs. Threads scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Scheduling algorithms – Categories and goals

- **Batch**
"business-world" applications, data analysis. Appropriate for non-preemptive
- **Interactive**
Many users that need responsiveness, requiring preemptive scheduling
- **Real-time**
for short-lived, short-cycle processes with hard/soft deadlines



... Let's discuss some scheduling algorithms for some of these categories [read others in book]...

Batch systems – scheduling

Algorithm	+	-
First-Come/First-Serve (non-preemptive)	Easy, fair	Possibly inefficient (esp. for I/O bound processes)
Shortest Job First (non-preemptive)	Optimal for turnaround	Starvation + need to know runtime
Shortest Remaining Time Next (preemptive)	New short jobs get good service	Starvation + need to know runtime

Example of inefficient first-come/first-serve scheduling

- Process 0 (CPU-bound): 1 I/O every 1 sec of computations, 1000 sec to finish
- Processes 1...1000 (I/O bound): need to perform 1000 I/Os
- FCFS: Processes 1...1000 get to perform 1 I/O every second. Hence, they end in 1000 seconds (>16 minutes)
- Preempting Process 0 every 10 ms, they could complete in 10 seconds...

Interactive Systems - scheduling

- Round-robin 
- Priority scheduling 
- Multiple queues
- Shortest process next
- Guaranteed scheduling
- Lottery scheduling 
- Fair-share scheduling

Round-Robin

- Quantum: time-interval during which the process can run
- Process still running at the end of the quantum? Preempt!
- Simple to implement (keep a list...)
- Challenge: what's the right quantum length?
 - Too short → high overhead
 - Too long → responsiveness (e.g., 50th process of a batch scheduled in round-robin with quantum 100ms waits 5 secs to start... what if it was the shortest I/O-bound of the 50 processes???)

Priority scheduling

- Not all processes are equally important, processes with higher priority should be prioritized
- Priorities:
 - Static (by OS or user)
 - Dynamic (by OS or user)
- Priority can be combined with round-robin → priority classes

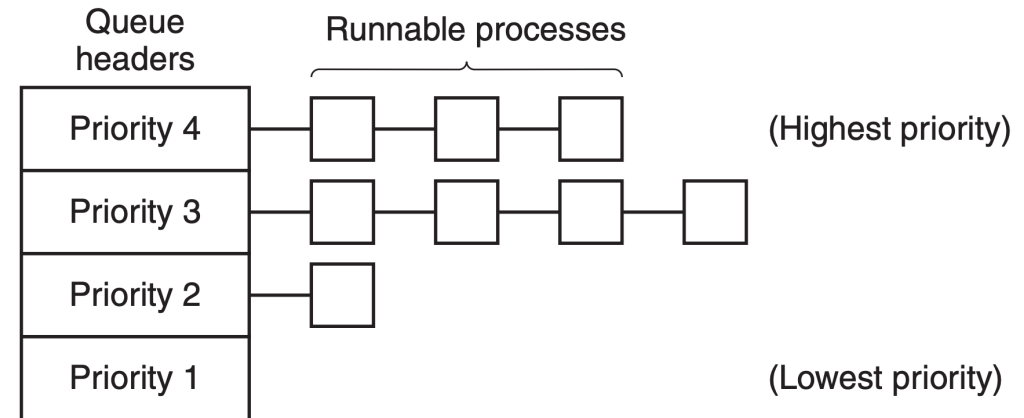


Figure 2-43. A scheduling algorithm with four priority classes.

...In the previous example (FCFS batch) I/O could have higher priority than CPU-bound...

Lottery scheduling

- Alternative to priority scheduling that still gives more resources to some processes rather than others
- Processes get “lottery tickets”.
- Next process to run is the one holding the next randomly chose ticket.
- Easier to map portions of resources to give to a process (i.e., portion of tickets to give) than with priority scheduling

Agenda

- Introduction
- Batch / Interactive / Real-time systems scheduling
- **Processes vs. Threads scheduling**
- Multiprocessor hardware
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

User-level vs. Kernel-level threads

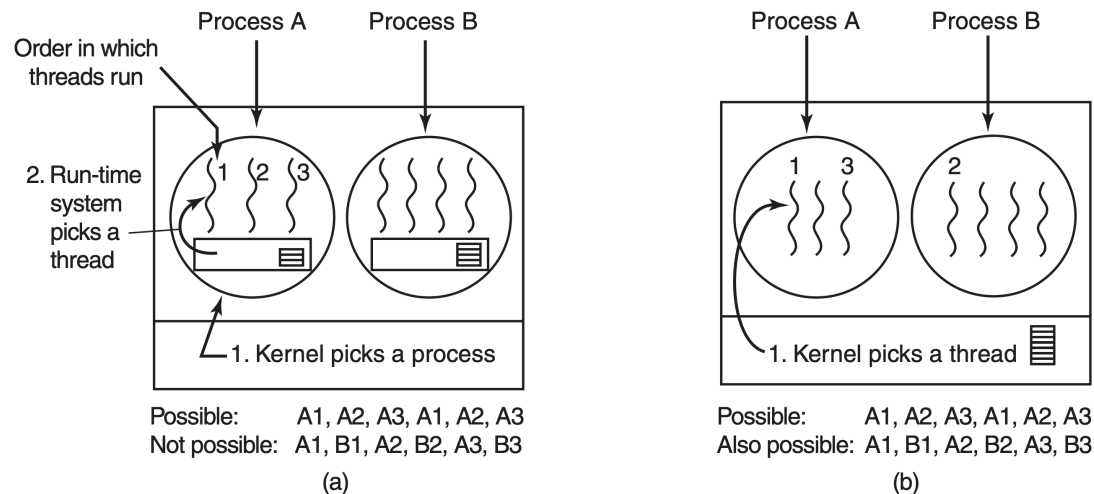


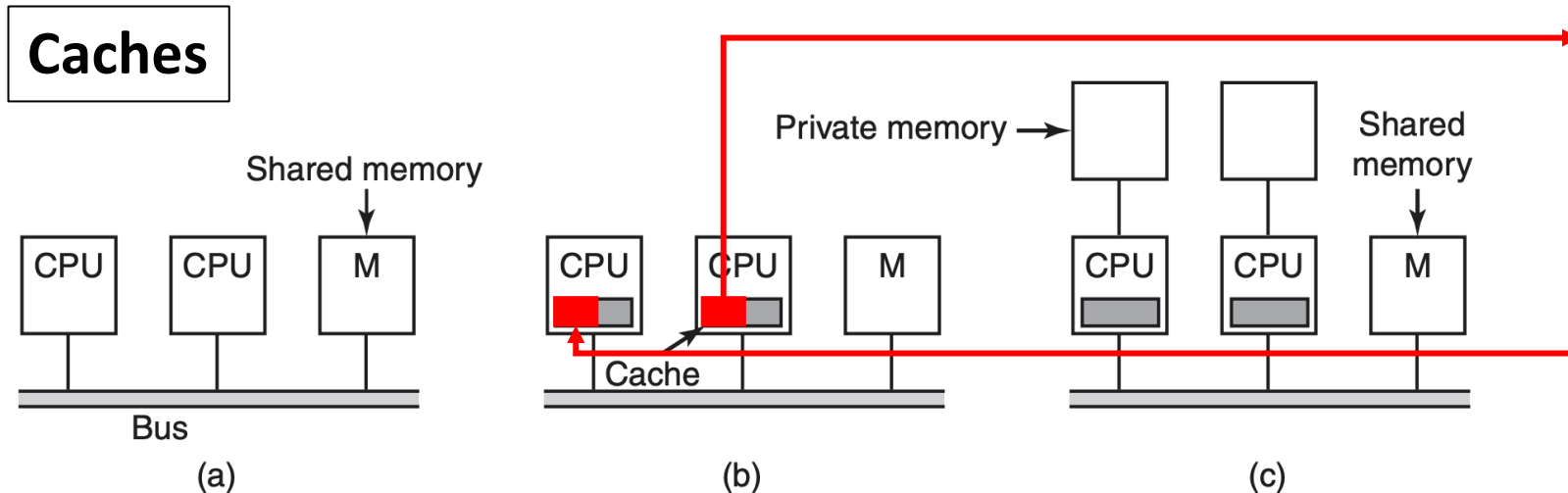
Figure 2-44. (a) Possible scheduling of user-level threads with a 50-msec process quantum and threads that run 5 msec per CPU burst. (b) Possible scheduling of kernel-level threads with the same characteristics as (a).

	User-level	Kernel-level
+	- Inter-quantum thread switch is extremely fast (no real context switch) - Can employ application specific scheduler	Process can keep running even if some of its thread perform I/O
-	A thread blocking on I/O means the entire process does	Thread switch costs more (but OS knows inter-process thread switch might cost more than intra-process one)

Agenda

- Introduction
- Batch / Interactive / Real-time systems scheduling
- Processes vs. Threads scheduling
- **Multiprocessor hardware**
 - Why does it complicate the matter?
- Multiprocessor scheduling
 - Time sharing
 - Space sharing
 - Gang scheduling

Multiprocessor hardware– Why complicated?



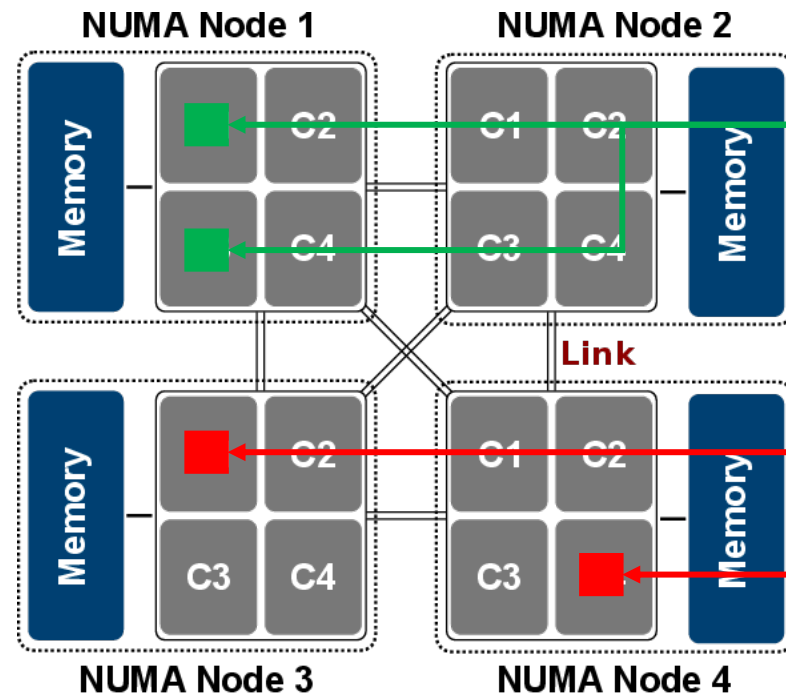
Suppose a thread has some data here and issues an I/O request. When later rescheduled, it might perform better on this CPU than on this one...

... but for that we need to keep track of more such information and make it part of the scheduling process.

Figure 8-2. Three bus-based multiprocessors. (a) Without caching. (b) With caching. (c) With caching and private memories.

Multiprocessor hardware– Why complicated?

NUMA architectures



Suppose two threads (producer/consumer) are scheduled at the same time...

Scheduling on the same socket will perform better than...

Scheduling on two different sockets

To complicate a bit further... how would the OS know 2 threads are producer/consumer?

Multiprocessor hardware– Why complicated?

Where to place the OS itself?

Each CPU its own OS

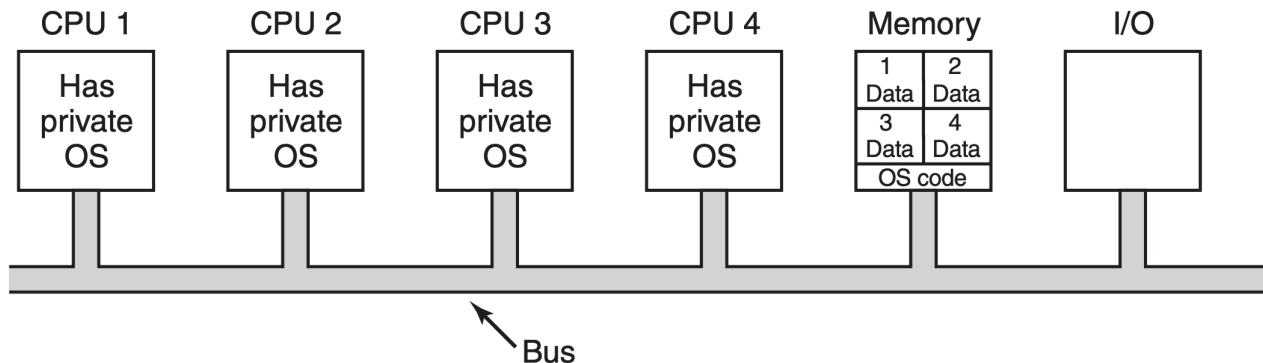


Figure 8-7. Partitioning multiprocessor memory among four CPUs, but sharing a single copy of the operating system code. The boxes marked Data are the operating system's private data for each CPU.

Might still be better than n separate computers

No sharing makes it simple, but also inefficient and possibly useless...

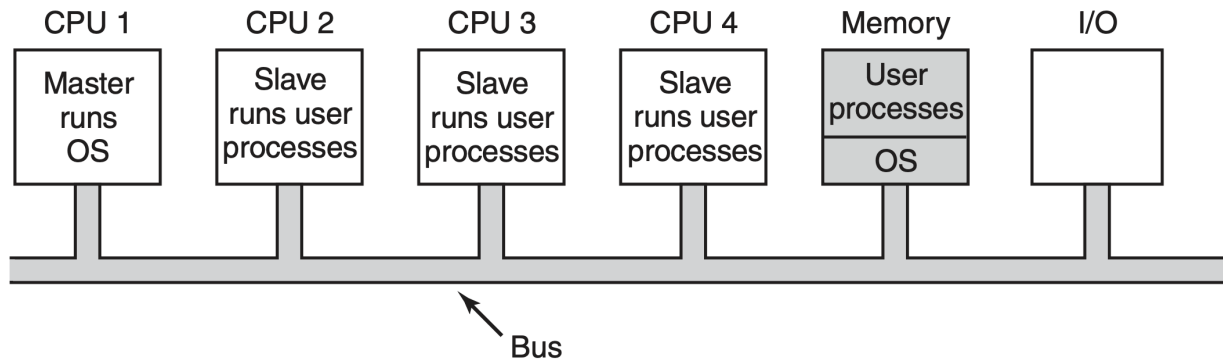
- Load can become imbalanced
- Data can become inconsistent (especially with buffers!)

Multiprocessor hardware– Why complicated?

Where to place the OS itself?

All system calls redirected to the Master CPU

Master-Slave



... easy to bottleneck ...

Figure 8-8. A master-slave multiprocessor model.

Multiprocessor hardware– Why complicated?

Where to place the OS itself?

Balances workload / resources

→ CONCURRENT ACCESS TO KERNEL!!!

Symmetric multiprocessors

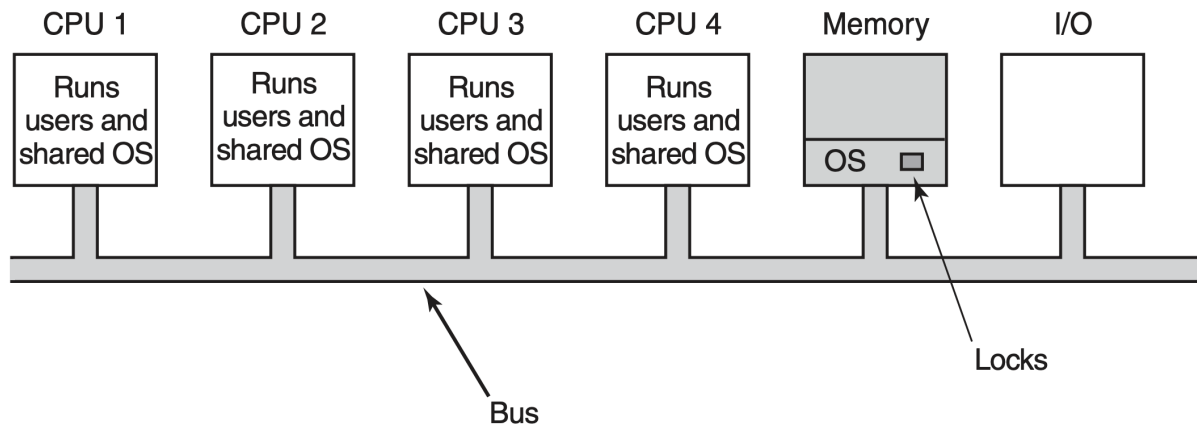


Figure 8-9. The SMP multiprocessor model.

2 threads could modify the same data structure at the same time.

Big lock? → Then it is basically master-slave

Critical regions / fine-grained parallelism? → better!
...but makes it hard to program (e.g., deadlocks...)

Agenda

- Introduction
- Batch / Interactive / Real-time systems scheduling
- Processes vs. Threads scheduling
- Multiprocessor hardware
 - Why does it complicate the matter?
- **Multiprocessor scheduling**
 - Time sharing
 - Space sharing
 - Gang scheduling

Time sharing

- Single system-wide data structure (or combination) for all ready threads

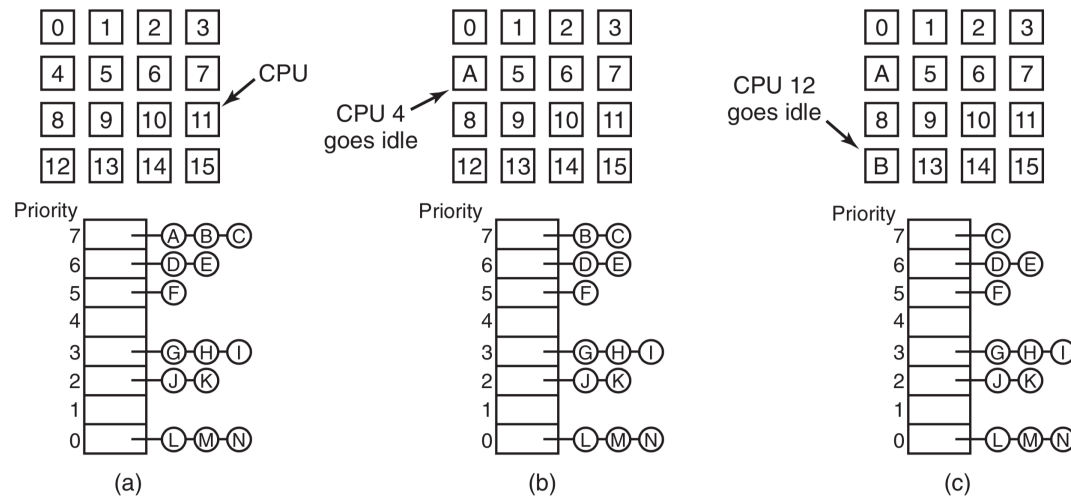


Figure 8-12. Using a single data structure for scheduling a multiprocessor.

- + Automatic load balancing
- Contention might bottleneck the system
- Still suffers from the “affinity problem”

Time sharing – two-level scheduling algorithm

- Each CPU has its collection of threads (assigned at creation time, in e.g. round-robin or least-loaded)
- Idle CPUs can still take threads from other CPUs if needed
- Benefits:
 - Load balancing
 - Cache affinity
 - Less contention

Space sharing

- When a set of related threads (e.g., from the same process) is created, the OS tries to schedule all of them at the same time (if enough CPUs are available).
- Thread issuing I/O still holds the CPU (inefficient...).

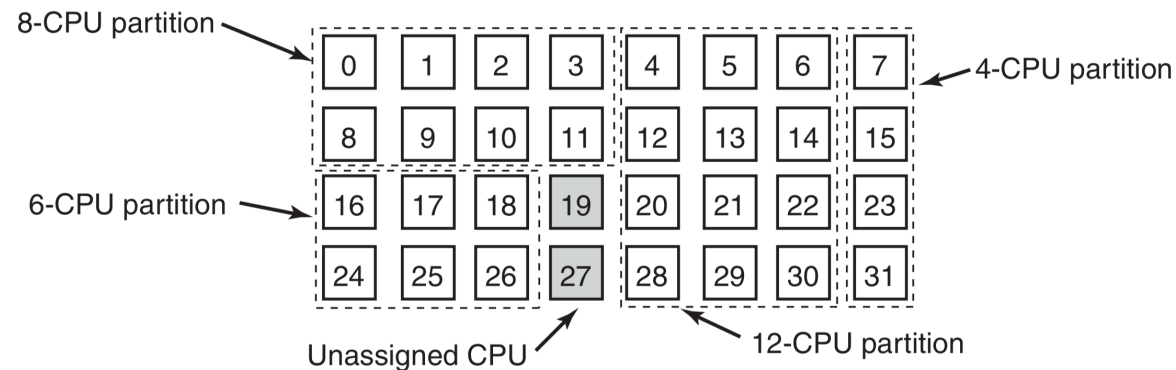
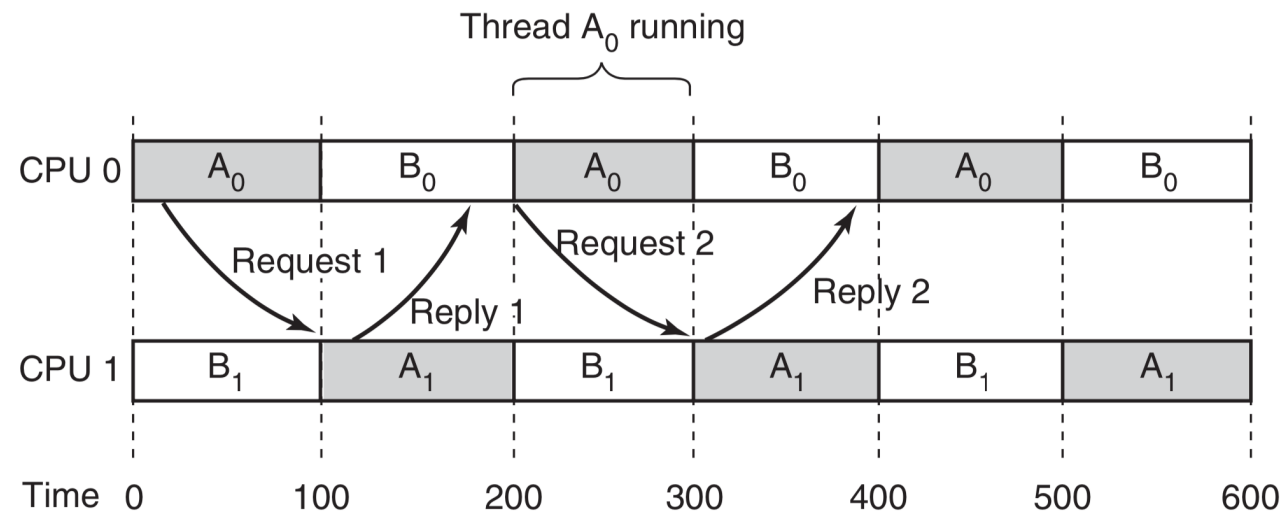


Figure 8-13. A set of 32 CPUs split into four partitions, with two CPUs available.

Gang scheduling

- Schedule both in time and space
- Can prevent problems like the one shown below:



Gang scheduling

- Groups of related threads are scheduled as a gang (with same quantum)
- All gang members run at once
- All gang members start / end their quantum together

Gang scheduling - example

		CPU					
		0	1	2	3	4	5
Time slot	0	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅
	1	B ₀	B ₁	B ₂	C ₀	C ₁	C ₂
	2	D ₀	D ₁	D ₂	D ₃	D ₄	E ₀
	3	E ₁	E ₂	E ₃	E ₄	E ₅	E ₆
	4	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅
	5	B ₀	B ₁	B ₂	C ₀	C ₁	C ₂
	6	D ₀	D ₁	D ₂	D ₃	D ₄	E ₀
	7	E ₁	E ₂	E ₃	E ₄	E ₅	E ₆

Figure 8-15. Gang scheduling.

Questions

32

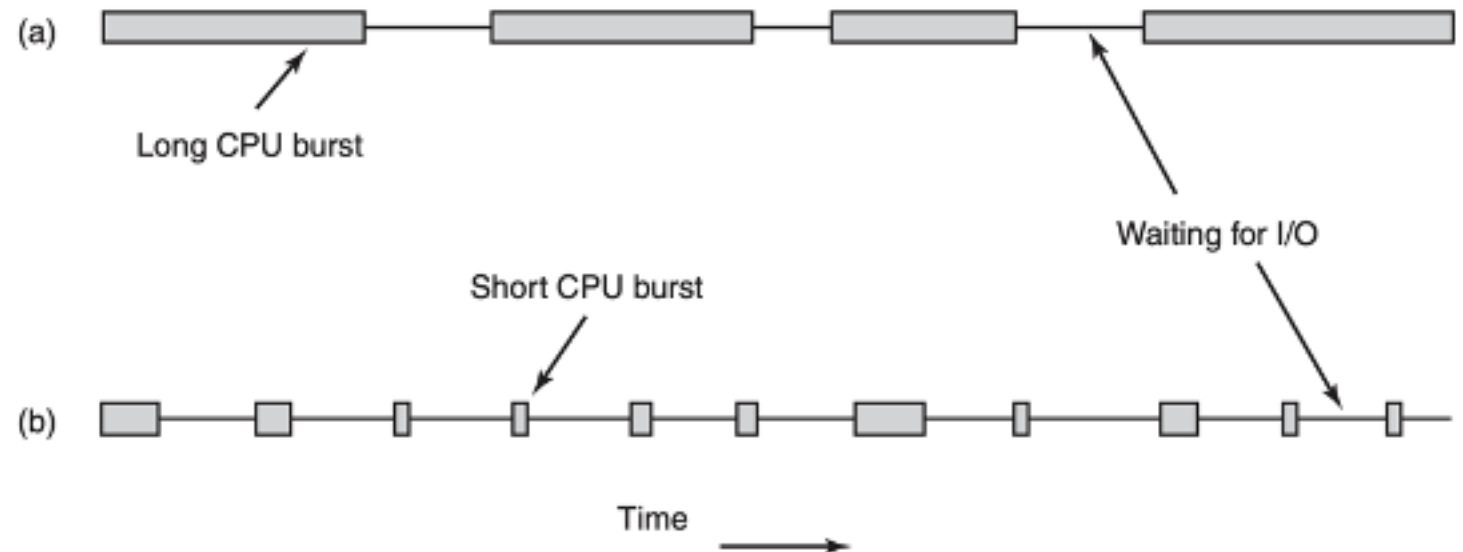
Vincenzo Gulisano

A slow I/O device makes a process accessing such a device I/O bound

1. True
2. False

Which of these 2 processes is CPU bound and I/O bound?

1. a is CPU, b is I/O
2. b is CPU, a is I/O
3. Is this one of your tricky questions?



The quantum duration in round-robin scheduling indicates how long the process will run

1. Yes
2. No

Kernel threads are better than user threads when a multithreaded process has one thread that is heavily I/O bounded

1. Yes
2. No

Kernel threads are better than user threads when a multithreaded process has one thread that is heavily I/O bounded

1. I said yes before but now I have doubts
2. I still believe the answer is yes

Setup: CPU bound procs that use 100% of their quantum, I/O-bound procs, priority + round-robin sched. To maximize fairness:

1. higher priority for CPU-bound procs
2. higher priority for I/O-bound procs